RFE

Functional Benefits of Automating Features using Recursive Feature Elimination (RFE), and L1 Regularization

Introduction to Feature Selection and Automation

All Data Scientist and Machine Learning Engineers knows that "Feature Engineering", specifically feature selection, is a crucial step in the machine learning and data science workflow that involves selecting the most relevant features from a derived dataset to train a model and take it further the model to the production environment. We have to do the proper exercise and carefully choose these features along with SEM's involvement and guidance; the end results would be enhanced model performance, improved interpretability, reduced overfitting, and highly optimised computational resources.

The goal is to use automated feature selection techniques to identify how it retains features that contribute the most to the model's outcome while removing irrelevant, redundant, or noisy data that could negatively impact performance.

Understanding the Automated Feature Selection

Automating feature selection techniques furthers this process by using algorithms and tools to streamline and accelerate it. This approach reduces manual effort, mitigates human biases, and ensures that the most informative features are chosen consistently across

large or complex datasets. Automated methods can handle high-dimensional data, making the process more scalable and efficient.

Automated feature selection can be categorized into three main types: Filter, Wrapper, and Embedded Methods. This article will explore Wrapper and Embedded Methods, their statistical metrics, and their understanding and implementation. We will also explore how they help machine learning models build worthy algorithms capable of identifying relevant and required features based on the processed dataset for the problem statement given.

Automated techniques help data scientists and engineers build better-performing models while saving time and effort, making them indispensable in modern machine-learning projects.

As we know, automatic feature selection uses algorithms to choose the best subset of features from the earlier article.

The following methods are designed to handle significant datasets with high-dimensional data and often provide faster, more objective results than manual selection. They are classified into Filter, Wrapper, and Embedded Methods.

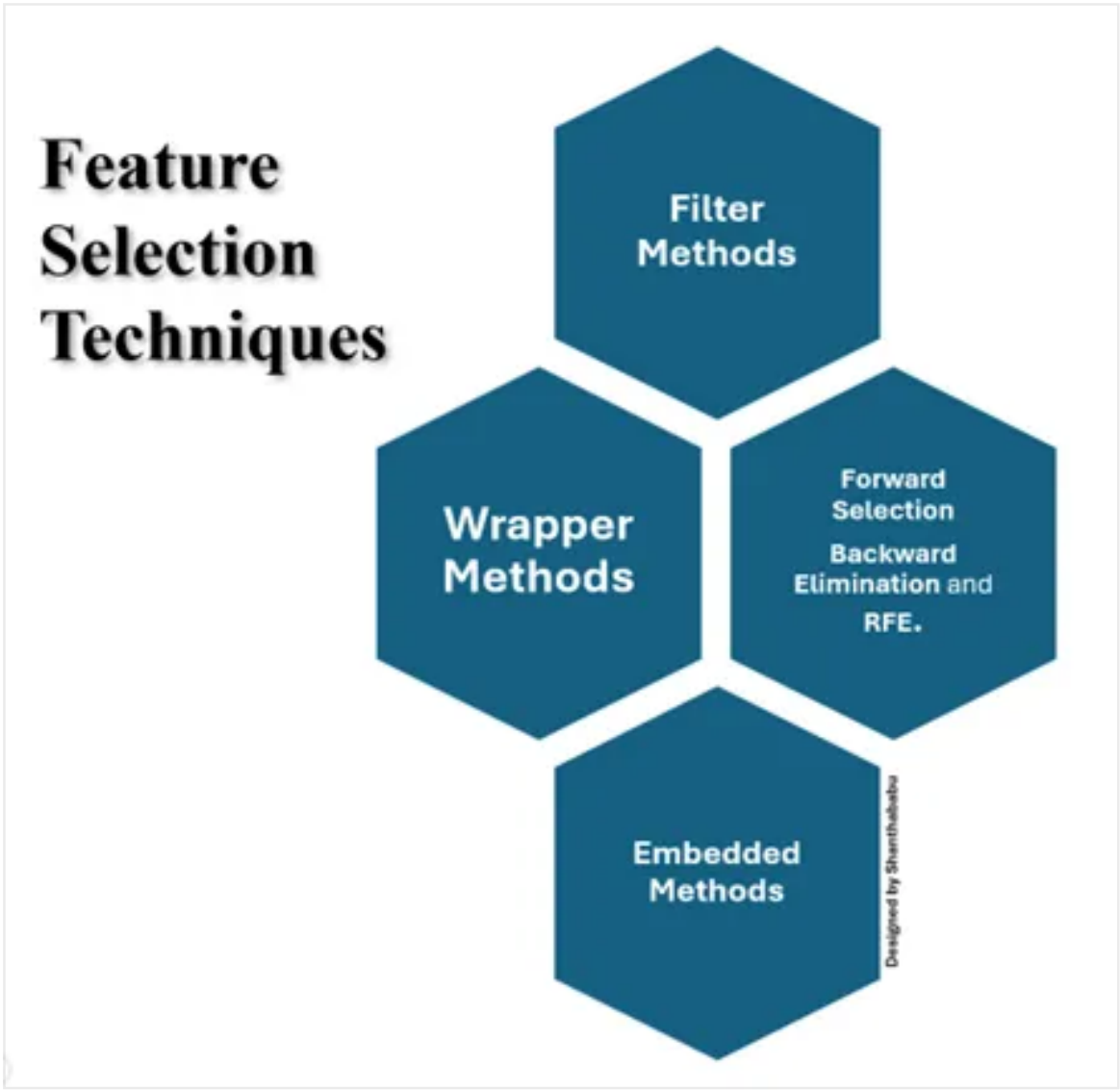

Figure 1: Feature Selection Techniques

Among these methods, start with “Filter Methods” to reduce dimensionality if the dataset is large. We can use “Embedded Methods” if model-based selection is preferred to balance performance and efficiency factors. Then, we can apply “Wrapper Methods” if feature interactions are crucial and computational resources allow it, particularly for smaller datasets. That’s the suggestion for a strategy for data scientists and ML engineers.

Comparative Study of Automated Feature Selection Methods

Automated Description and Techniques: Each method is unique. Let’s understand each process and the

techniques available.

Method	Description	Techniques
Filter Methods	This is the process of evaluating each feature in the given dataset, independently of any model, using statistical measures to assess relevance.	• Correlation Coefficient • Chi-square Test • Mutual Information • Variance Threshold
Wrapper Methods	This evaluates the subsets of features by training a model on each subset and assessing performance and observation on each stage	• Forward Selection • Backward Elimination • Recursive Featur Elimination (RFE)
Embedded Methods	This process incorporates feature selection within the model training process, using model mechanisms to identify importance.	• Lasso Regression (L1) • Decision Tree-base models • Elastic Net

Table 1: Comparative Study of Automated Feature Selection Methods

Automated Methods — Advantages and Disadvantages: Each method has its own advantages and disadvantages. Let's explore each one quickly.

Method	Advantages	Disadvantages
Filter Methods	• It is a really Fast way to implement using simple coding and is computationally inexpensive. • It is Ideal for high-dimensional data	• It Ignores feature interaction characteristics. • It overlooks combinations of features that work well together.
Wrapper Methods	• It exhibits better model performance than Filter Methods	• Computationally expensive • Time-consuming process • Occasionally risk of overfitting.
Embedded Methods	• Efficient feature selection and tight integration with model training • Good balance of cost and performance aspects than other methods	• In some scenarios, selected features may be model-dependent • In some scenarios, it may not be generalized across different models.

Table 2: Automated Methods — Advantages and Disadvantages

Automated Techniques: Although the three methods have unique advantages and various techniques, each method's fitment differs based on the scenarios it is best for.

Method	Best For
Filter Methods	• Large datasets with many features • When computational speed is an essential aspect • When building simple models • When considering the cost-effective model
Wrapper Methods	• Small to medium dataset size • Where feature interactions are essential • Where high computational resources are available.
Embedded Methods	• High-dimensional dataset • Where model interpretability and relevance are essential factors • When the model-specific solutions are acceptable.

Table 3: Automated Methods — Best fit

Auto Feature Selection

Let's discuss "Auto Feature Selection Tools" in Python. Libraries like scikit-learn in Python offer automated feature selection tools such as SelectKBest, Recursive Feature Elimination (RFE) and LassoCV.

- SelectKBest: Select the top k features based on a scoring function (e.g., Chi-square, ANOVA).
- Recursive Feature Elimination (RFE): Iteratively fits the model and removes less essential features.
- LassoCV: Performs feature selection while optimizing the regularization strength in Lasso Regression.

As I promised initially, let's implement the Wrapper Methods in this article.

Wrapper Methods

It uses a set of techniques to evaluate different subsets of features by training a model and assessing

performance; because of this, methods require more computation effort.
Of course, we can get better capture feature interactions than filter methods. It has three techniques to demonstrate its capabilities: Forward Selection, Backward Elimination, and RFE.

- Forward Selection starts with an empty model and keeps adding the features one by one, evaluates the performance, observes the outcome, and, based on the results, decides which feature improves the model the most at each step.
- Backward Elimination starts with all features in the initial evaluation and eliminates them one by one. It then evaluates the performance, observes the outcome, and removes the least significant feature based on the results at each step.
- Recursive Feature Elimination (RFE) method trains the model with all features from the given dataset and iteratively removes the least essential features based on model performance. It is often used with algorithms that provide feature importance, like decision trees.

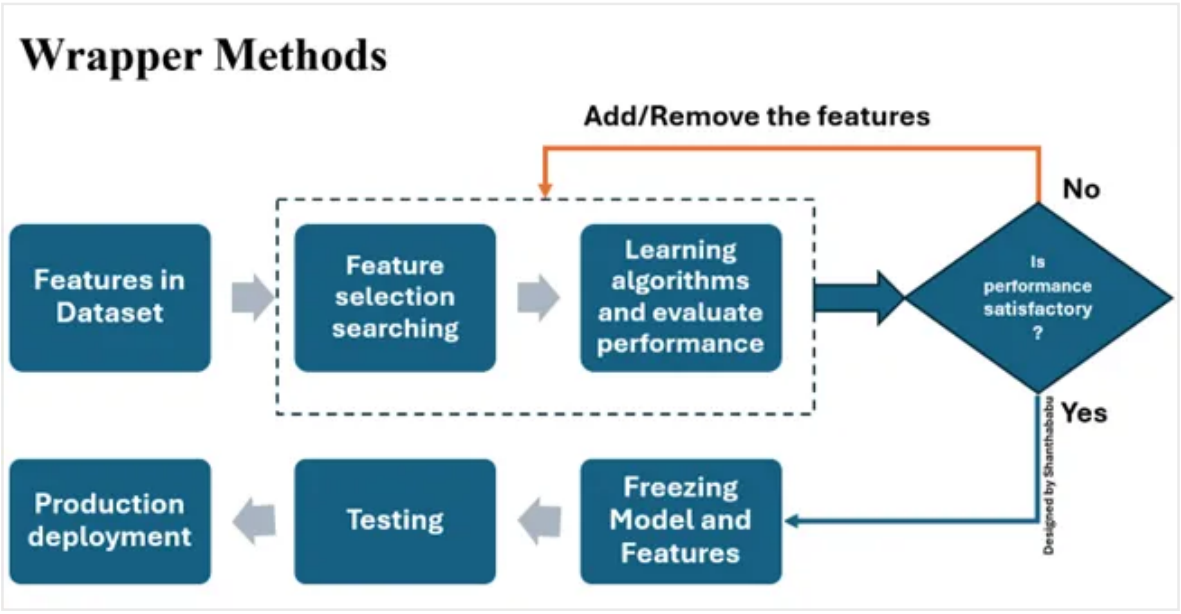

Figure 2: Feature Selection Techniques — Wrapper method process

```python
import pandas as pd
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
import warnings
warnings.filterwarnings('ignore')

# Load a winequality dataset
X = df_winequality.drop("quality", axis=1)
y = df_winequality["quality"]

def forward_selection(X, y, model, scoring='accuracy', cv=5):

    selected_features = []
    remaining_features = list(X.columns)
    best_score = 0

    while remaining_features:
        scores_with_candidates = []
        for feature in remaining_features:
```

```
            # Test the current set of selected features plus
the candidate feature
            candidate_features = selected_features +
[feature]
            X_subset = X[candidate_features]
            score = cross_val_score(model, X_subset, y,
scoring=scoring, cv=cv).mean()
            scores_with_candidates.append((score, feature))

        # Select the feature with the highest score
        scores_with_candidates×sort(reverse=True)
        best_new_score, best_candidate =
scores_with_candidates[0]

        # Stop if no improvement
        if best_new_score <= best_score:
            break

        # Update the best score and add the best
candidate to selected features
        best_score = best_new_score
        selected_features.append(best_candidate)
        remaining_features.remove(best_candidate)
        print(f"Selected feature: {best_candidate} with
score: {best_new_score}")

    return selected_features

# Instantiate the model
model =
LogisticRegression(max_iter=10,solver='liblinear')

# Perform forward selection
selected_features = forward_selection(X, y, model)
print("Final selected features:", selected_features)
```

```

Output
Selected feature: alcohol with score:
0.551620297805642 7
Selected feature: volatile acidity with score:
0.558483542319749 2
Selected feature: citric acid with score:
0.566626175548589 3
Selected feature: sulphates with score:
0.572884012539185
Selected feature: chlorides with score:
0.575384012539185
Selected feature: residual sugar with score:
0.576626175548589 3
Final selected features: ['alcohol', 'volatile acidity', 'citric acid', 'sulphates', 'chlorides', 'residual sugar']
Observation
Key Insights from the Output
Feature Selection Process:

- We have implemented forward selection to iteratively choose features based on their contribution to the model's performance.
- Each iteration adds the feature from the given dataset (winequality), increasing the cross-validated accuracy score.

Feature Scores:

- Based on the code implementation, the selection process begins with alcohol, which achieved the highest initial score of 0.5516.
- Subsequent features
- Volatile Acidity
- Citric Acid
```

- Sulphates
- Chlorides
- Residual Sugar

Were added one at a time, each providing an incremental improvement in model performance. Each chosen feature leads to a higher score than previous combinations, indicating that these features contribute significantly to model accuracy.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE

# Load a sample dataset (you can replace this with your own dataset)
data = load_wine()
X = data.data
y = data.target



# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Instantiate a Logistic Regression model
model = LogisticRegression(max_iter=1000, solver='liblinear')

# Set up RFE to select the top 5 features
n_features_to_select = 5
rfe = RFE(estimator=model, n_features_to_select=n_features_to_select)
```

```
# Fit RFE
rfe.fit(X_train, y_train)

# Get the selected features
selected_features = rfe.support_
feature_ranking = rfe.ranking_

# Print the selected features and their rankings
print("Selected Features (True means selected):",
selected_features)
print("Feature Ranking:", feature_ranking)

# Get names of selected features if using a dataframe
selected_feature_names = [data.feature_names[i] for i
in range(len(selected_features)) if selected_features[i]]
print("Selected Feature Names:",
selected_feature_names)

# Evaluate model performance on the test set using
only selected features
X_train_selected = rfe.transform(X_train)
X_test_selected = rfe.transform(X_test)
model.fit(X_train_selected, y_train)
score = model.score(X_test_selected, y_test)
print(f"Model accuracy with selected features:
{score:.4f}")
```

Output

```
Selected Features (True means selected):
[False False  True False False False  True False False
True  True  True  False]
Feature Ranking: [7 5 1 3 8 4 1 6 2 1 1 1 9]
Selected Feature Names: ['ash', 'flavanoids',
'color_intensity', 'hue', 'od280/od315_of_diluted_wines']
Model accuracy with selected features: 0.8704
```

Observation

Selected Features (True means selected):

- This array shows a True or False for each feature in the dataset, indicating whether RFE selected each feature.
- True means the feature was selected as one of the top 5 essential features, while False means it was not.
- For example, [False, False, True, False, False, False, True, False, False, True, True, True, False] indicates that the 3rd, 7th, 10th, 11th, and 12th features were selected.

Feature Ranking:

- Each feature's ranking shows how important RFE considered it, with 1 indicating selected features and higher numbers indicating lower importance.
- For example:
- Features ranked 1 are selected: ash, flavanoids, color_intensity, hue, and od280/od315_of_diluted_wines.
- The feature ranking 9 (the highest) was deemed the least important.

Selected Feature Names:

- The selected features (ash, flavanoids, color_intensity, hue, and od280/od315_of_diluted_wines) are printed here by name.
- The top 5 features that RFE identified as most significant for the logistic regression model in predicting the target.

Model Accuracy with Selected Features:

- The model's accuracy using only the selected features is 0.8704.
- This accuracy score indicates how well the model performs on the test data when using just the selected subset of features rather than the entire set.
- A relatively high score here (0.8704) suggests that the selected features are well-suited to the model and can make accurate predictions, simplifying the model without sacrificing much accuracy.

Conclusion

Automating feature selection through Recursive Feature Elimination (RFE) and L1 Regularization offers substantial functional benefits in developing machine learning models.

These techniques efficiently streamline selecting the most relevant features, reducing manual effort, computational cost, and model complexity.

By focusing on the features that significantly influence the model's predictions, these automated methods improve the model's accuracy, interpretability, and generalization capability.

In our implementations:

- Forward Selection in wrapper methods selected features based on their incremental contribution to accuracy, resulting in a robust feature subset that improves model performance in stages.
- Recursive Feature Elimination (RFE) effectively reduced the dataset to the most impactful features, preserving only those essential for predicting outcomes while achieving

a high model accuracy of 0.8704.

Both methods confirm that automated feature selection simplifies model training and produces models optimized for high-dimensional data. By automating feature selection, data scientists can focus on refining model accuracy and robustness while minimizing the risk of overfitting and enhancing the model's deployment readiness. Thus, automated selection is an indispensable part of a scalable and efficient data science workflow.

Thanks for reading this article.